嵌入式 Qt 开发实例



正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。



扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：	正点原子 linux 团队	正点原子 linux 团队	2025.04.16

目录

前言 5
第一章 基于 Qt 的 LED 点亮实现 7
1.1 LED 硬件原理及驱动 8
1.2 点亮 LED 实战 8
第二章 Qt 赋能 USB 摄像头的摄影功能开发实战 11
2.1 USB 格式与分辨率查询 12
2.2 测试 USB 摄像头 15
2.3 USB 摄像头拍照实战 15

前言

本文档作为 Qt 进阶文档，主要面向已有一定 Qt 基础，期望进一步提升开发技能的开发者。若您尚未熟悉 Qt 开发基础内容，建议先查阅《嵌入式 Qt 开发基础文档》，以构建扎实的知识根基。在接下来的内容里，笔者将引领大家深入探索 Qt 开发的实际应用，通过一系列精心挑选的典型例子，展现 Qt 在不同场景下的强大功能和开发技巧，助力大家实现从基础到进阶的跨越。

关于作者：

B 站：https://space.bilibili.com/474103963?spm_id_from=333.788.0.0 // 查看更多作品

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 基于 Qt 的 LED 点亮实现

在当今科技飞速发展的时代，嵌入式系统的应用无处不在，而利用软件开发工具对硬件设备进行控制是其中的关键环节。Qt 作为一款功能强大且广泛应用的跨平台应用程序开发框架，以其简洁易用的接口和高效的开发效率，成为众多开发者的首选。

在第一章 “基于 Qt 的 LED 点亮实现” 中，我们将开启一段基础而又意义重大的探索之旅。LED 作为一种常见且重要的电子元件，广泛应用于状态指示、信息显示等场景。通过学习如何基于 Qt 实现 LED 的点亮，你将初步掌握软件与硬件交互的基本原理和方法。

本章会先介绍 Qt 开发环境的搭建，确保你拥有一个良好的开发起点。接着，详细剖析 LED 点亮的原理，包括硬件连接和软件逻辑。我们会逐步引导你编写代码，从初始化设置到最终实现 LED 的点亮操作。每一步都配有清晰的解释和示例，让你能够轻松理解和掌握。无论你是嵌入式开发的新手，还是希望进一步拓展技能的开发者，本章都将为你提供宝贵的知识和实践经验。

1.1 LED 硬件原理及驱动

相关介绍请查看网盘 A 盘资料/10、用户手册/开发文档/Linux 驱动开发指南 LED 子系统章节。

1.2 点亮 LED 实战

通过上一章节的了解。我们知道“/sys/class/leds/work/brightness”这个文件用于控制对应 LED 设备的亮度。通过向这个文件写入不同的值，可以控制 LED 的亮灭。通常，这个文件接受的数值范围是从 0 和 1，0 表示关闭 LED，1 表示 LED 点亮。并且了解到“/sys/class/leds/work/trigger”这个文件是控制 LED 的触发模式，想要控制 LED 的工作方式需要设置为“none”，表示手动控制模式，LED 的亮灭完全由用户通过写入 brightness 文件来控制，不受其他系统事件影响。

好了那么我们知道是这些文件控制 LED 的亮灭了，那么我们用 Qt 写个按钮来控制 LED 亮灭吧！

例程路径为网盘 A 盘资料/01、程序源码/Qt 开发实例源码/01_qt_led_control（仅适用开发板，不要在 Ubuntu 上运行，Ubuntu 上没有 LED 设备，看不到效果的！）。

代码赏析：

```
/**************************************************************
Copyright © Deng Zhimao Co., Ltd. 2021-2030. All rights reserved.
* @brief        widget.cpp
* @author       Deng Zhimao
* @email        dengzhimao@alientek.com/1252699831@qq.com
* @date         2025-04-10
* @link         http://www.alientek.com
* @LICENSE      GPLV3
**************************************************************/
#include "widget.h"
#include "ui_widget.h"
#include <QDebug>

Widget::Widget(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::Widget)
{
    ui->setupUi(this);
    // 设置全屏
    this->showFullScreen();
    // 修改为手动控制，当然你可以用 QFile 写入“none”
    system("echo \"none\" > /sys/class/leds/work/trigger");
    // 设置 led 控制文件
    m_ledFile.setFileName("/sys/class/leds/work/brightness");
    // 程序初始化时，先读取 LED 状态
    if (m_ledFile.open(QIODevice::ReadOnly)) {
```

```
27          setState(m_ledFile.readAll().simplified());
28          m_ledFile.close(); // 打开需要关闭
29      }
30      ui->pushButton->setCheckable(true);// 设置可被选中
31  }
32
33  Widget::~Widget()
34  {
35      delete ui;
36  }
37
38  QString Widget::state() const
39  {
40      return m_state;
41  }
42
43  void Widget::setState(const QString &newState)
44  {
45      if (m_state == newState)
46          return;
47      m_state = newState;
48      if (bool(m_state == "1") == State::On) {
49          ui->pushButton->setText("关灯");
50          // 设置不同的颜色
51          ui->pushButton->setStyleSheet("QPushButton{background-color:
rgb(239, 41, 41)}");
52      } else {
53          ui->pushButton->setText("开灯");
54          ui->pushButton->setStyleSheet("QPushButton{background-color:
rgb(136, 138, 133)}");
55      }
56      emit stateChanged();
57  }
58
59  void Widget::on_pushButton_clicked(bool checked)
60  {
61      // 被选中时，开灯
62      if (checked) {
63          if(m_ledFile.open(QIODevice::ReadWrite)) {
64              m_ledFile.write("1");
65              ui->pushButton->setText("关灯");
66              ui->pushButton->setStyleSheet("QPushButton{background-
color: rgb(239, 41, 41)}");
```

```
67              m_ledFile.close();
68          }
69      } else {
70          if(m_ledFile.open(QIODevice::ReadWrite)) {
71              m_ledFile.write("0");
72              ui->pushButton->setText("开灯");
73              ui->pushButton->setStyleSheet("QPushButton{background-
color: rgb(136, 138, 133)}");
74              m_ledFile.close();
75          }
76      }
77  }
```

第 22 行，设置 LED 为手动控制触发模式。

第 59~77 行，为 LED 控制亮灭重要代码，通过点击按钮，对 m_ledFile 文件写 0/1 字符来控制亮灭。

程序界面如下：



第二章 Qt 赋能 USB 摄像头的摄影功能开发实战

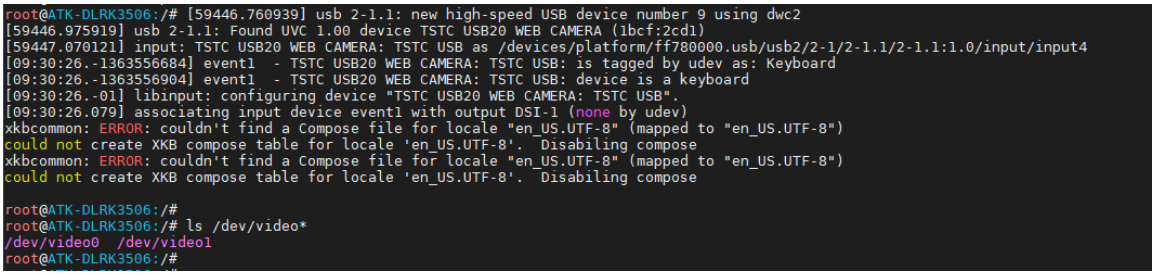
在当今数字化浪潮中，USB 摄像头作为一种常见的图像采集设备，广泛应用于安防监控、视频会议、图像识别等诸多领域。然而，要将其无缝融入到 Qt 应用程序开发框架中，充分挖掘并拓展其摄影功能，并非易事。

本章节《Qt 赋能 USB 摄像头的摄影功能开发实战》聚焦于此，旨在为开发者们提供一站式的解决方案。紧接着，通过详实的代码示例，手把手带您搭建起基于 Qt 的摄像头操作环境，涵盖摄像头设备的初始化、参数配置，如分辨率、帧率、等调校，确保拍摄效果达到最优。不仅如此，还将深入到图像捕捉的关键时刻，传授如何在 Qt 事件循环中精准触发拍照指令，高效处理并存储拍摄所得的图像数据，无论是保存为常见的 JPEG、PNG 格式，都能轻松驾驭。真正实现 Qt 与 USB 摄像头完美融合，开启摄影功能开发的全新征程。

2.1 USB 格式与分辨率查询

在底板的 USB 接口处插上 USB 摄像头，底板必须接上 12V 电源适配器，否则供电不足！终端打印信息如下，同时产生 video*设备节点，一般 video0 是摄像头出图节点。

```
ls /dev/video*
```



我们可以使用 v4l2-ctl 查看 USB 摄像头支持的分辨率与格式。注 USB 摄像头请自备，一般都支持 YUYV 采集。

```
v4l2-ctl -d /dev/video0 --list-formats-ext
```

终端输出信息如下，可以看到笔者的摄像头支持的格式与帧率非常多，有些 USB 摄像头没有那么多分辨率支持，请注意。

```
root@ATK-DLRK3506:/# v4l2-ctl -d /dev/video0 --list-formats-ext
ioctl: VIDIOC_ENUM_FMT
        Type: Video Capture

        [0]: 'MJPG' (Motion-JPEG, compressed)
                Size: Discrete 3840x2160
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.050s (20.000 fps)
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 2592x1944
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.050s (20.000 fps)
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 2048x1536
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.050s (20.000 fps)
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.200s (5.000 fps)
```

Size: Discrete 1600x1200
Interval: Discrete 0.033s (30.000 fps)
Interval: Discrete 0.040s (25.000 fps)
Interval: Discrete 0.050s (20.000 fps)
Interval: Discrete 0.067s (15.000 fps)
Interval: Discrete 0.100s (10.000 fps)
Interval: Discrete 0.200s (5.000 fps)
Size: Discrete 1920x1080
Interval: Discrete 0.033s (30.000 fps)
Interval: Discrete 0.040s (25.000 fps)
Interval: Discrete 0.050s (20.000 fps)
Interval: Discrete 0.067s (15.000 fps)
Interval: Discrete 0.100s (10.000 fps)
Interval: Discrete 0.200s (5.000 fps)
Size: Discrete 1280x960
Interval: Discrete 0.033s (30.000 fps)
Interval: Discrete 0.040s (25.000 fps)
Interval: Discrete 0.050s (20.000 fps)
Interval: Discrete 0.067s (15.000 fps)
Interval: Discrete 0.100s (10.000 fps)
Interval: Discrete 0.200s (5.000 fps)
Size: Discrete 1280x720
Interval: Discrete 0.033s (30.000 fps)
Interval: Discrete 0.040s (25.000 fps)
Interval: Discrete 0.050s (20.000 fps)
Interval: Discrete 0.067s (15.000 fps)
Interval: Discrete 0.100s (10.000 fps)
Interval: Discrete 0.200s (5.000 fps)
Size: Discrete 800x600
Interval: Discrete 0.033s (30.000 fps)
Interval: Discrete 0.040s (25.000 fps)
Interval: Discrete 0.050s (20.000 fps)
Interval: Discrete 0.067s (15.000 fps)
Interval: Discrete 0.100s (10.000 fps)
Interval: Discrete 0.200s (5.000 fps)
Size: Discrete 640x480
Interval: Discrete 0.033s (30.000 fps)
Interval: Discrete 0.040s (25.000 fps)
Interval: Discrete 0.050s (20.000 fps)
Interval: Discrete 0.067s (15.000 fps)
Interval: Discrete 0.100s (10.000 fps)
Interval: Discrete 0.200s (5.000 fps)
Size: Discrete 320x240

```
                Interval: Discrete 0.033s (30.000 fps)
                Interval: Discrete 0.040s (25.000 fps)
                Interval: Discrete 0.050s (20.000 fps)
                Interval: Discrete 0.067s (15.000 fps)
                Interval: Discrete 0.100s (10.000 fps)
                Interval: Discrete 0.200s (5.000 fps)
        [1]: 'YUYV' (YUYV 4:2:2)
                Size: Discrete 3840x2160
                        Interval: Discrete 1.000s (1.000 fps)
                Size: Discrete 2592x1944
                        Interval: Discrete 1.000s (1.000 fps)
                Size: Discrete 2048x1536
                        Interval: Discrete 0.333s (3.000 fps)
                Size: Discrete 1600x1200
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 1280x960
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 800x600
                        Interval: Discrete 0.050s (20.000 fps)
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.050s (20.000 fps)
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.200s (5.000 fps)
                Size: Discrete 320x240
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.050s (20.000 fps)
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.200s (5.000 fps)
```

2.2 测试 USB 摄像头

首先先用 GST 指令播放摄像头画面，检查 USB 摄像头是否正常。可以看到如下画面，USB 摄像头已经播放了。

```
gst-launch-1.0 v4l2src device=/dev/video0 ! autovideosink
```



2.3 USB 摄像头拍照实战

通过前面的准备工作，确保 USB 摄像头工作正常了再到本小节。让我们直接开始吧，源码注释十分详细，请查看。界面设计了一个摄像头预览框，一个拍照预览框，注连续拍照会覆盖前一张照片，用户可以自行设计。

例程路径为网盘 A 盘资料/01、程序源码/Qt 开发实例源码/02_qcamera。

代码赏析：

```
1
/**************************************************************************
2   Copyright © Deng Zhimao Co., Ltd. 2021-2030. All rights reserved.
3   * @brief        widget.cpp
4   * @author       Deng Zhimao
5   * @email        dengzhimao@alientek.com/1252699831@qq.com
```

```
6   * @date         2025-04-16
7   * @link         http://www.alientek.com
8   * @LICENSE      GPLV3
9   ********************************************************************/
10  #include "widget.h"
11  #include <QDebug>
12  #include <QVBoxLayout>
13  #include <QPushButton>
14  #include <QLabel>
15  #include <QFont>
16  #include <QCamera>
17  #include <QVideoWidget>
18  #include <QCameraImageCapture>
19  Widget::Widget(QWidget *parent)
20      : QWidget(parent)
21  {
22      // 全屏显示
23      this->showFullScreen();
24      // 请根据各自的摄像头节点填写,RK3506 一般就是 video0
25     QCamera *m_qcamera = new QCamera("/dev/video0", this);
26
27      if (!m_qcamera) {
28          qDebug() << "摄像头初始化失败！";
29      }
30
31      // 字体大小
32      QFont font;
33      font.setPixelSize(30);
34
35      //  布局
36      QVBoxLayout *layout = new QVBoxLayout(this);
37      layout->setAlignment(Qt::AlignCenter);
38
39      QLabel *titleLabel = new QLabel();
40      titleLabel->setText("正点原子Qt赋能USB摄像头的摄影功能开发实战");
41      titleLabel->setFixedSize(this->width(), 100);
42      titleLabel->setFont(font);
43      titleLabel->setAlignment(Qt::AlignCenter);
44      layout->addWidget(titleLabel);
45
46      QCameraViewfinderSettings settings;
47      // 设置分辨率
48      settings.setResolution(640, 480);
```

```
49      m_qcamera->setViewfinderSettings(settings);
50
51      QVideoWidget *m_videoWidget = new QVideoWidget();
52      // 设置大小
53      m_videoWidget->setFixedSize(this->width(), 480);
54
55      layout->addWidget(m_videoWidget);
56
57      // 拍照按钮
58      QPushButton *captureButton = new QPushButton();
59      captureButton->setFixedSize(this->width(), 100);
60      captureButton->setText("拍照");
61
62      captureButton->setFont(font);
63
64      layout->addWidget(captureButton);
65
66      QLabel *captureLabel = new QLabel();
67      captureLabel->setFixedSize(this->width(), 480);
68
69      // 显示拍照
70      layout->addWidget(captureLabel);
71
72      // 设置视频输出
73      m_qcamera->setViewfinder(m_videoWidget);
74      m_qcamera->start();
75
76      m_videoWidget->show();
77
78      QCameraImageCapture *imageCapture = new
QCameraImageCapture(m_qcamera, this);
79
80      // 连接拍照按钮的点击信号到拍照槽函数
81      QObject::connect(captureButton, &QPushButton::clicked,
[imageCapture,captureLabel]() {
82          // 拍照,拍照路径默认为 userdata 路径下
83          imageCapture->capture("/userdata/imageCapture.jpg");
84      });
85
86      QObject::connect(imageCapture, &QCameraImageCapture::imageSaved,
[captureLabel]() {
87          // 拍照完成
88          QImage image("/userdata/imageCapture.jpg");
```

```
89          if (!image.isNull()) {
90              captureLabel->setPixmap(QPixmap::fromImage(image));
91          }
92      });
93  }
94
95  Widget::~Widget()
96  {
97
98  }
```

程序运行界面如下。下图上部分为摄像头预览框，下部分为拍照显示框，中间为拍照按钮，点击可拍照，照片保存路径默认为/userdata/imageCapture.jpg。



因板子外设有限，目前只写了上面两个例程。更多例程请查看 Qt 开发基础例程（较基础）及 Qt 综合例程（难度高）。